R. Uday kumar Reddy (192424190) **C3 Lab Experiments**

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

1.

```
def max_min(arr, low, high):
    if low == high:
        return arr[low], arr[low]
    if high == low + 1:
        if arr[low] < arr[high]:
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]
    mid = (low + high) // 2
    min1, max1 = max_min(arr, low, mid)
    min2, max2 = max_min(arr, mid + 1, high)
    return min(min1, min2), max(max1, max2)
arr = [5,7,3,4,9,12,6,2]
minimum, maximum = max_min(arr, 0, len(arr)-1)
print("Minimum =", minimum)
print("Maximum =", maximum)
```

```
Minimum = 2
Maximum = 12
```

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

2.

```
def max_min_sorted(arr):
    minimum = arr[0]
    maximum = arr[-1]
    return minimum, maximum
arr = [2,4,6,8,10,12,14,18]
minimum, maximum = max_min_sorted(arr)
print("Minimum =", minimum)
print("Maximum =", maximum)
```

```
Minimum = 2
Maximum = 18
```

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

3.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = [31,23,35,27,11,21,15,28]
merge_sort(arr)
print(arr)
```

```
[11, 15, 21, 23, 27, 28, 31, 35]
```

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

4.

```
count = 0
def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            count += 1
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = [12,4,78,23,45,67,89,1]
merge_sort(arr)
print("Sorted Array :", arr)
print("Comparisons :", count)
```

```
Sorted Array : [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons : 16
```

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

5.

```
count = 0
def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            count += 1
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1

            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = [12,4,78,23,45,67,89,1]
merge_sort(arr)
print("Sorted Array :", arr)
print("Comparisons :", count)
```

```
Sorted Array : [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons : 16
```

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

6.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = []
    middle = []
    right = []
    for x in arr:
        if x < pivot:
            left.append(x)
        elif x == pivot:
            middle.append(x)
        else:
            right.append(x)
    sorted_arr = quick_sort(left) + middle + quick_sort(right)
    print(sorted_arr)
    return sorted_arr
arr = [19,72,35,46,58,91,22,31]
print("Final Sorted Array:")
print(quick_sort(arr))
```

```
Final Sorted Array:
[31, 35]
[19, 22, 31, 35]
[19, 22, 31, 35, 46]
[72, 91]
[19, 22, 31, 35, 46, 58, 72, 91]
[19, 22, 31, 35, 46, 58, 72, 91]
```

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

7.

```
def binary_search(arr, key):
    low = 0
    high = len(arr)-1
    count = 0
    while low <= high:
        count += 1
        mid = (low + high)//2
        if arr[mid] == key:
            return mid + 1, count
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, count
arr = [5,10,15,20,25,30,35,40,45]
position, comparisons = binary_search(arr,20)

print("Position =", position)
print("Comparisons =", comparisons)
```

```
Position = 4
Comparisons = 4
```

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

8.

```
def binary_search(arr, key):
    low = 0
    high = len(arr)-1
    while low <= high:
        mid = (low + high)//2
        print("Low =", low,
              "High =", high,
              "Mid =", mid,
              "Value =", arr[mid])
        if arr[mid] == key:
            return mid + 1
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = [3,9,14,19,25,31,42,47,53]
print("Position =", binary_search(arr,31))
```

```
Low = 0 High = 8 Mid = 4 Value = 25
Low = 5 High = 8 Mid = 6 Value = 42
Low = 5 High = 5 Mid = 5 Value = 31
Position = 6
```

9. Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

9.

```
import math
def k_closest(points, k):
    points.sort(key=lambda p: math.sqrt(p[0]**2 + p[1]**2))
    return points[:k]
points = [[1,3],[-2,2],[5,8],[0,1]]
k = 2
print(k_closest(points,k))
```

```
[[0, 1], [-2, 2]]
```

10. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

10.

```
def four_sum(A,B,C,D):
    count=0
    for a in A:
        for b in B:
            for c in C:
                for d in D:
                    if a+b+c+d==0:
                        count+=1
    return count
A=[1,2]
B=[-2,-1]
C=[-1,2]
D=[0,2]
print(four_sum(A,B,C,D))
```

```
2
```

11. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

11.

```
def median_of_medians(arr,k):
    arr.sort()
    return arr[k-1]
arr=[12,3,5,7,19]
print(median_of_medians(arr,2))
```

```
5
```

12. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

12.

```
def kth_smallest(arr,k):
    arr.sort()
    return arr[k-1]
arr=[23,17,31,44,55,21,20,18,19,27]
k=5
print(kth_smallest(arr,k))
```

```
21
```

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.

13.

```
from itertools import combinations
def closest_subset(arr,target):
    best=[]
    best_sum=0
    diff=float("inf")
    n=len(arr)
    for r in range(n+1):
        for subset in combinations(arr,r):
            s=sum(subset)
            if abs(target-s)<diff:
                diff=abs(target-s)
                best=subset
                best_sum=s
    return best,best_sum
arr=[45,34,4,12,5,2]
target=42
subset,total=closest_subset(arr,target)
print(subset)
print(total)
```

```
(34, 4, 5)
43
```

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

14.

```
from itertools import combinations
def exact_sum(arr,target):
    n=len(arr)
    for r in range(n+1):
        for subset in combinations(arr,r):
            if sum(subset)==target:
                return True
    return False
arr=[3,34,4,12,5,2]
print(exact_sum(arr,15))
```

```
True
```

15 Given two 2×2 Matrices A and B

A=(1 7 B=(1 3

3 5) 7 5)

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that C=A×B.

15.

```
def strassen(A,B):
    a,b=A[0]
    c,d=A[1]
    e,f=B[0]
    g,h=B[1]
    p1=a*(f-h)
    p2=(a+b)*h
    p3=(c+d)*e
    p4=d*(g-e)
    p5=(a+d)*(e+h)
    p6=(b-d)*(g+h)
    p7=(a-c)*(e+f)
    c11=p5+p4-p2+p6
    c12=p1+p2
    c21=p3+p4
    c22=p1+p5-p3-p7
    return [[c11,c12],[c21,c22]]
A=[[1,7],
   [3,5]]
B=[[1,3],
   [7,5]]
print(strassen(A,B))
```

```
[[50, 38], [38, 34]]
```

16 Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product Z=X x Y

16.

```
def karatsuba(x,y):
    if x<10 or y<10:
        return x*y
    n=max(len(str(x)),len(str(y)))
    m=n//2
    high1=x//10**m
    low1=x%10**m
    high2=y//10**m
    low2=y%10**m
    z0=karatsuba(low1,low2)
    z1=karatsuba((low1+high1),(low2+high2
    z2=karatsuba(high1,high2)
    return z2*10**(2*m)+(z1-z2-z0)*10**m+
x=1234
y=5678
print(karatsuba(x,y))
```

```
7006652
```